$mod ::= \textbf{MODULE}(mname, dep\ ..., defn\ ...)$
$M ::= mod\ ...$
$mname ::= name$
$dep ::= mname@ph$
$deps ::= dep\ ...$
$defn ::= \langle dname@ph, kind, ast\rangle$
$defns ::= defn\ ...$
$dname ::= name$
$kind ::= \textbf{VALUE} \mid \textbf{MACRO}$
$ph ::= number$

$binding ::= \langle ph, mname, dname\rangle$
$bindings ::= binding\ ...$

$\Sigma ::= \{cell\ ...\}$
$cell ::= \langle binding, kind, val\rangle$
$\quad \mid \langle mname@ph\rangle$

$s\text{-}exp ::= name \mid (\ s\text{-}exp\ ...\ )$
$name ::=$ a token such as `x`, `clock`, or `lambda`

$ast ::= val$
$\quad \mid \textbf{REF}(binding)$
$\quad \mid var$
$\quad \mid \textbf{APP}(ast, ast, ...)$
$val ::= func$
$\quad \mid \textbf{QUOTE}(name)$
$\quad \mid \textbf{SYNTAX}(stx)$
$\quad \mid \textbf{UNBOUND}$
$func ::= \textbf{LAMBDA}(var\ ..., ast)$
$var ::= \textbf{VAR}(name)$

$stx ::= id \mid (\ stx\ ...\ )$
$id ::= \textbf{STX}(name, bindings)$

$body ::= stx@ph$

---

cmodule : $s\text{-}exp\ M \to M$

cmodule$[\![(\texttt{module}\ mname\ mname_{init}\ s\text{-}exp\ ...\ ), M]\!]$
$=$ cbody$[\![body\ ..., mname,\ mname_{init}@0, \varepsilon, M, \Sigma_{init}]\!]$
subject to visit$[\![mname_{init}, 0, M, \varnothing]\!] = \Sigma_{init}$,
import$[\![mname_{init}, 0, M]\!] = bindings$,
wrap$[\![s\text{-}exp, bindings]\!]@0\ ... = body\ ...$

---

cbody : $body\ ...\ mname\ dep\ ...\ defn\ ...\ M\ \Sigma \to M$

cbody$[\![\varepsilon, mname, deps, defns, M, \Sigma]\!] = M\big[mname \longleftarrow \textbf{MODULE}(mname, deps, defns)\big]$

cbody$[\![(\ id_{beg}\ stx\ ...\ )\ @ph\ body_{rest}\ ..., mname, deps, defns, M, \Sigma]\!]$
$=$ cbody$[\![stx@ph\ ...\ body_{rest}\ ..., mname, deps, defns, M, \Sigma]\!]$
subject to resolve$[\![id_{beg}, ph]\!] = \langle ph, \texttt{base}, \texttt{begin}\rangle$

cbody$[\![(\ id_{bfs}\ stx\ ...\ )\ @ph\ body_{rest}\ ..., mname, deps, defns, M, \Sigma]\!]$
$=$ cbody$[\![stx@ph{+}1\ ...\ body_{rest}\ ..., mname, deps, defns, M, \Sigma]\!]$
subject to resolve$[\![id_{bfs}, ph]\!] = \langle ph, \texttt{base}, \texttt{begin-for-syntax}\rangle$

cbody$[\![(\ id_{def}\ id_{new}\ stx\ )\ @ph\ body_{rest}\ ..., mname, deps, defn\ ..., M, \Sigma]\!]$
$=$ cbody$[\![body_{new}\ ..., mname, deps, defn\ ...\ defn_{new}, M, \Sigma_{new}]\!]$
subject to resolve$[\![id_{def}, ph]\!] = \langle ph, \texttt{base}, \texttt{define}\rangle$,
strip$[\![id_{new}]\!] = dname$, cexpr$[\![stx, ph, \varepsilon, \Sigma]\!] = ast$,
$\langle dname@ph, \textbf{VALUE}, ast\rangle = defn_{new}$,
$\langle ph, mname, dname\rangle = binding_{new}$,
add$[\![body_{rest}, binding_{new}]\!]\ ... = body_{new}\ ...$,
maybe-eval$[\![ph, ast, \Sigma]\!] = val$,
$\Sigma\big[binding_{new} \longleftarrow \langle \textbf{VALUE}, val\rangle\big] = \Sigma_{new}$

cbody⟦ (id_{defstx} id_{new} stx) @ph $body_{rest}$ ..., $mname$, $deps$, $defn$ ..., M, Σ⟧
= cbody⟦$body_{new}$ ..., $mname$, $deps$, $defn$ ... $defn_{new}$, M, Σ_{new}⟧
subject to resolve⟦id_{defstx}, ph⟧ = ⟨ph, `base`, `define-syntax`⟩,
strip⟦id_{new}⟧ = $dname$, cexpr⟦stx, ph+1, ε, Σ⟧ = ast,
⟨$dname$@ph, **MACRO**, ast⟩ = $defn_{new}$,
⟨ph, $mname$, $dname$⟩ = $binding_{new}$,
add⟦$body_{rest}$, $binding_{new}$⟧ ... = $body_{new}$ ...,
eval⟦ast, Σ⟧ = val, Σ[$binding_{new}$ ⟵ ⟨**MACRO**, val⟩] = Σ_{new}

cbody⟦ (id_{macro} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦stx_{new}@ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
subject to resolve⟦id_{macro}, ph⟧ = $binding$,
Σ($binding$) = ⟨**MACRO**, $func$⟩,
eval⟦**APP**($func$, **SYNTAX**((id_{macro} stx ...))), Σ⟧ = **SYNTAX**(stx_{new})

cbody⟦ (id_{req} stx_{in}) @ph $body_{rest}$ ..., $mname$, dep ..., $defns$, M, Σ⟧
= cbody⟦$body_{new}$ ..., $mname$, dep ... dep_{new}, $defns$, M, Σ_{new}⟧
subject to resolve⟦id_{req}, ph⟧ = ⟨ph, `base`, `require`⟩,
parse-req⟦stx_{in}, ph⟧ = ⟨$mname_{in}$, ph_{Δ}⟩,
visit⟦$mname_{in}$, ph_{Δ}, M, Σ⟧ = Σ_{new},
import⟦$mname_{in}$, ph_{Δ}, M⟧ = $bindings_{new}$,
add⟦$body_{rest}$, $bindings_{new}$⟧ ... = $body_{new}$ ...,
$mname_{in}$@ph_{Δ} = dep_{new}

cbody⟦ (id_{mod} id_{sub} id_{init} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M_{new}, Σ⟧
subject to resolve⟦id_{mod}, ph⟧ = ⟨ph, `base`, `module`⟩,
strip⟦id_{sub}⟧ = $mname_{sub}$, strip⟦id_{init}⟧ = $mname_{init}$,
cmodule⟦ (`module` $mname_{sub}$ $mname_{init}$ strip⟦stx⟧ ...) , M⟧ = M_{new}

cbody⟦ (id_{mod} id_{sub} id_{init} stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cmodule⟦ (`module` $mname_{sub}$ $mname_{init}$ strip⟦stx⟧ ...) , M_{new}⟧
subject to resolve⟦id_{mod}, ph⟧ = ⟨ph, `base`, `module*`⟩,
strip⟦id_{sub}⟧ = $mname_{sub}$, strip⟦id_{init}⟧ = $mname_{init}$,
cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧ = M_{new}

cbody⟦ (id_{mod} id_{sub} ( ) stx ...) @ph $body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧
= cbody⟦shift⟦stx, 0–ph⟧@0 ..., $mname_{sub}$, $mname$@0–ph, ε, M_{new}, Σ_{init}⟧
subject to resolve⟦id_{mod}, ph⟧ = ⟨ph, `base`, `module*`⟩,
strip⟦id_{sub}⟧ = $mname_{sub}$,
cbody⟦$body_{rest}$ ..., $mname$, $deps$, $defns$, M, Σ⟧ = M_{new},
visit⟦$mname$, 0–ph, M_{new}, ∅⟧ = Σ_{init}

---

cexpr : stx ph env Σ → ast

cexpr⟦id, ph, env, Σ⟧ = var
subject to env-lookup⟦env, **VAR**(strip⟦id⟧)⟧ = var

$\text{cexpr}[\![id, ph, env, \Sigma]\!] = \textbf{REF}(binding)$
subject to $\text{resolve}[\![id, ph]\!] = binding$

$\text{cexpr}[\![(\ id_{lambda}\ (\ id\ ...\ )\ stx\ )\ , ph, name_{env}\ ..., \Sigma]\!]$
$= \textbf{LAMBDA}(\textbf{VAR}(\text{strip}[\![id]\!])\ ..., \text{cexpr}[\![stx, ph, \textbf{VAR}(\text{strip}[\![id]\!])\ ...\ name_{env}\ ..., \Sigma]\!])$
subject to $\text{resolve}[\![id_{lambda}, ph]\!] = \langle ph, \texttt{base}, \texttt{lambda}\rangle$

$\text{cexpr}[\![(\ id_{quote}\ stx\ )\ , ph, env, \Sigma]\!] = \textbf{QUOTE}(\text{strip}[\![stx]\!])$
subject to $\text{resolve}[\![id_{quote}, ph]\!] = \langle ph, \texttt{base}, \texttt{quote}\rangle$

$\text{cexpr}[\![(\ id_{qstx}\ stx\ )\ , ph, env, \Sigma]\!] = \textbf{SYNTAX}(stx)$
subject to $\text{resolve}[\![id_{qstx}, ph]\!] = \langle ph, \texttt{base}, \texttt{syntax}\rangle$

$\text{cexpr}[\![(\ id_{macro}\ stx\ ...\ )\ , ph, env, \Sigma]\!] = \text{cexpr}[\![stx_{new}, ph, env, \Sigma]\!]$
subject to $\text{resolve}[\![id_{macro}, ph]\!] = binding,$
$\Sigma(binding) = \langle \textbf{MACRO}, func\rangle,$
$\text{eval}[\![\textbf{APP}(func, \textbf{SYNTAX}(\ (\ id_{macro}\ stx\ ...\ )\ ))), \Sigma]\!] = \textbf{SYNTAX}(stx_{new})$

$\text{cexpr}[\![(\ stx\ ...\ )\ , ph, env, \Sigma]\!] = \textbf{APP}(\text{cexpr}[\![stx, ph, env, \Sigma]\!], ...)$

$\text{cexpr}[\![id, ph, env, \Sigma]\!] = \textbf{UNBOUND}$

$\text{eval} : ast\ \Sigma \rightarrow val$

$\text{eval}[\![val, \Sigma]\!] = val$

$\text{eval}[\![\textbf{REF}(binding), \Sigma]\!] = val$
subject to $\Sigma(binding) = \langle \textbf{VALUE}, val\rangle$

$\text{eval}[\![\textbf{APP}(ast, ast_{arg}, ...), \Sigma]\!] = \text{eval}[\![ast_{body}[var\ ... \leftarrow val\ ...], \Sigma]\!]$
subject to $\text{eval}[\![ast, \Sigma]\!] = \textbf{LAMBDA}(var\ ..., ast_{body}),$
$\text{eval}[\![ast_{arg}, \Sigma]\!]\ ... = val\ ...$

$\text{maybe-eval} : ph\ ast\ \Sigma \rightarrow val$

$\text{maybe-eval}[\![0, ast, \Sigma]\!] = \textbf{UNBOUND}$

$\text{maybe-eval}[\![ph, ast, \Sigma]\!] = \text{eval}[\![ast, \Sigma]\!]$

$\text{ast-shift} : ast\ ph \rightarrow ast$

$\text{ast-shift}[\![var, ph]\!] = var$

$\text{ast-shift}[\![\textbf{LAMBDA}(var\ ..., ast), ph]\!] = \textbf{LAMBDA}(var\ ..., \text{ast-shift}[\![ast, ph]\!])$

$\text{ast-shift}[\![\textbf{QUOTE}(name), ph]\!] = \textbf{QUOTE}(name)$

ast-shift⟦**APP**$(ast, ...), ph$⟧ = **APP**(ast-shift⟦ast, ph⟧, ...)

ast-shift⟦**REF**$(\langle ph_{ref}, mname, dname\rangle), ph$⟧ = **REF**$(\langle ph_{ref}+ph, mname, dname\rangle)$

ast-shift⟦**SYNTAX**$(stx), ph$⟧ = **SYNTAX**(shift⟦stx, ph⟧)

ast-shift⟦**UNBOUND**$, ph$⟧ = **UNBOUND**

---

visit : $mname\ ph\ M\ \Sigma \rightarrow \Sigma$

visit⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = Σ
subject to $\Sigma(mname@ph_{\Delta})$ = **READY**

visit⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma_{deps}$⟧
subject to $M(mname)$ = **MODULE**$(mname, deps, defn\ ...)$,
$\Sigma\left[mname@ph_{\Delta} \leftarrow \textbf{READY}\right] = \Sigma_{init}$,
visit*⟦$deps, ph_{\Delta}, M, \Sigma_{init}$⟧ = Σ_{deps}

---

visit* : $deps\ ph\ M\ \Sigma \rightarrow \Sigma$

visit*⟦$\varepsilon, ph_{\Delta}, M, \Sigma$⟧ = Σ

visit*⟦$mname@ph\ dep\ ..., ph_{\Delta}, M, \Sigma$⟧ = visit*⟦$dep\ ..., ph_{\Delta}, M, \Sigma_{new}$⟧
subject to visit⟦$mname, ph_{\Delta}+ph, M, \Sigma$⟧ = Σ_{new}

---

visit-body : $defns\ name\ ph\ \Sigma \rightarrow \Sigma$

visit-body⟦$\varepsilon, mname, ph_{\Delta}, \Sigma$⟧ = Σ

visit-body⟦$\langle dname@ph_{def}$, **VALUE**$, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
= visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
subject to $ph_{\Delta}+ph_{def} = 0$

visit-body⟦$\langle dname@ph_{def}, kind, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
= visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
subject to pmax⟦$-1, ph_{\Delta}+ph_{def}$⟧ = -1

visit-body⟦$\langle dname@ph_{def}, kind, ast\rangle\ defn\ ..., mname, ph_{\Delta}, \Sigma$⟧
= visit-body⟦$defn\ ..., mname, ph_{\Delta}, \Sigma_{new}$⟧
subject to $ph_{\Delta}+ph_{def} = ph$, eval⟦shift⟦$ast, ph_{\Delta}$⟧, Σ⟧ = val,
$\Sigma\left[\langle ph, mname, dname\rangle \leftarrow \langle kind, val\rangle\right] = \Sigma_{new}$

---

run : $mname\ ph\ M\ \Sigma \rightarrow \Sigma$

run⟦$mname, ph_{\Delta}, M, \Sigma$⟧ = Σ
subject to $\Sigma(mname@ph_{\Delta})$ = **READY**

run⟦$mname, ph_\Delta, M, \Sigma$⟧ = run-body⟦$defn$ ..., $mname, ph_\Delta, \Sigma_{deps}$⟧
subject to $M(mname)$ = **MODULE**($mname, deps, defn$ ...),
$\Sigma[mname@ph_\Delta \leftarrow$ **READY**$] = \Sigma_{init}$,
run*⟦$deps, ph_\Delta, M, \Sigma_{init}$⟧ = Σ_{deps}

---

run* : $deps\ ph\ M\ \Sigma \rightarrow \Sigma$

run*⟦$\varepsilon, ph_\Delta, M, \Sigma$⟧ = Σ

run*⟦$mname@ph\ dep$ ..., ph_Δ, M, Σ⟧ = run*⟦dep ..., $ph_\Delta, M, \Sigma_{new}$⟧
subject to run⟦$mname, ph_\Delta{+}ph, M, \Sigma$⟧ = Σ_{new}

---

run-body : $defns\ name\ ph\ \Sigma \rightarrow \Sigma$

run-body⟦$\varepsilon, mname, ph_\Delta, \Sigma$⟧ = Σ

run-body⟦$\langle dname@ph_{def}$, **VALUE**, $ast\rangle\ defn$ ..., $mname, ph_\Delta, \Sigma$⟧
= run-body⟦$defn$ ..., $mname, ph_\Delta, \Sigma_{new}$⟧
subject to $ph_\Delta{+}ph_{def} = 0$, eval⟦shift⟦ast, ph_Δ⟧, Σ⟧ = val,
$\Sigma[\langle 0, mname, dname\rangle \leftarrow \langle$**VALUE**, $val\rangle] = \Sigma_{new}$

run-body⟦$defn_{skip}\ defn$ ..., $mname, ph_\Delta, \Sigma$⟧
= run-body⟦$defn$ ..., $mname, ph_\Delta, \Sigma$⟧

---

wrap : $s\text{-}exp\ bindings \rightarrow stx$

wrap⟦$name, bindings$⟧ = **STX**($name, bindings$)

wrap⟦($s\text{-}exp$ ...), $bindings$⟧ = (wrap⟦$s\text{-}exp, bindings$⟧ ...)

---

strip : $stx \rightarrow s\text{-}exp$

strip⟦**STX**($name, bindings$)⟧ = $name$

strip⟦(stx ...)⟧ = (strip⟦stx⟧ ...)

---

add-to-stx : $stx\ bindings \rightarrow stx$

add-to-stx⟦**STX**($name, binding$ ...), $binding_{new}$ ...⟧
= **STX**($name, binding$ ... $binding_{new}$ ...)

add-to-stx⟦(stx ...), $bindings$⟧ = (add-to-stx⟦$stx, bindings$⟧ ...)

---

add : $body\ bindings \rightarrow body$

add⟦$stx@ph, bindings$⟧ = add-to-stx⟦$stx, bindings$⟧$@ph$

---

resolve : $id\ ph \rightarrow maybe\text{-}binding$

resolve⟦$\mathbf{STX}(dname, binding_{pre} \ldots\ binding\ binding_{post} \ldots), ph$⟧ = $binding$
subject to $binding = \langle ph, mname, dname \rangle$

resolve⟦id, ph⟧ = #f

---

shift : $stx\text{-}or\text{-}ast\ ph \rightarrow stx\text{-}or\text{-}ast$

shift⟦$\mathbf{STX}(name, \langle ph, mname, dname \rangle \ldots), ph_{shift}$⟧
= $\mathbf{STX}(name, \langle ph{+}ph_{shift}, mname, dname \rangle \ldots)$

shift⟦($stx \ldots$), ph_{shift}⟧ = (shift⟦stx, ph_{shift}⟧ ...)

shift⟦ast, ph_{shift}⟧ = ast-shift⟦ast, ph_{shift}⟧

---

parse-req : $stx\ ph \rightarrow \langle name, ph \rangle$

parse-req⟦id_{in}, ph⟧ = ⟨strip⟦id_{in}⟧, ph⟩

parse-req⟦($id_{fs}\ stx_{in}$), ph⟧ = parse-req⟦$stx_{in}, ph{+}1$⟧
subject to strip⟦id_{fs}⟧ = `for-syntax`

parse-req⟦($id_{ft}\ stx_{in}$), ph⟧ = parse-req⟦$stx_{in}, ph{-}1$⟧
subject to strip⟦id_{ft}⟧ = `for-template`

---

import : $mname\ ph_{\Delta}\ M \rightarrow bindings$

import⟦$mname, ph_{\Delta}, M$⟧ = $\langle ph_{\Delta}{+}ph_{def}, mname, dname \rangle \ldots$
subject to $M(mname) = \mathbf{MODULE}(mname, dep \ldots, defn \ldots)$,
$defn \ldots = \langle dname@ph_{def}, kind, ast \rangle \ldots$